Linda Meets Unix

Wm Leler, Cogent Research, Inc.

As microprocessors become more and more powerful, it becomes increasingly possible to build inexpensive multiple processor machines with far greater computational power than the fastest traditional supercomputers. Although such hardware is relatively easy to build, it has a well-earned reputation for being difficult to program. Many of the problems associated with programming multiprocessor computers are due to the use of inappropriate parallel programming models that were developed for vastly different purposes. To realize the promise of parallel computing, innovative new tools are required that will make these new parallel computers significantly easier to program.

The models of parallel computing in common use today are largely inherited from operating systems. Operating systems have long supported concurrent execution of processes through multiprogramming, which is really pseudo parallelism because multiple processes take turns executing rather than actually running in parallel. With multiple processor machines becoming the norm rather than the exception, operating systems are beginning to support multiprocessing (true parallelism). Unfortunately, these two forms of parallelism are fundamentally different from each other. Multiprogramming allows several independent programs or users to share a single processor, while multiprocessing allows a single program to use multiple processors. Put another way, multiprogramming increases hardware utilization, possibly at the expense of execution speed of individual programs, while multiprocessing increases the execution speed of individual programs.

A system-level version of the Linda high-level parallel software paradigm is used as the basis of the QIX operating system, which supports both multiprocessing and multiprogramming while retaining Unix compatibility.

As the mechanisms originally developed to support pseudo parallelism are pressed into service to support true parallelism, their limitations are becoming more and more apparent. For example, even though both forms of parallelism incorporate the concept of a process, the "heavyweight," application-oriented form of process developed during the days of multiprogramming is largely unsuitable for multiprocessing because process creation is too complicated and expensive.

The shift from pseudo parallelism to true parallelism involves taking the multiple processes of a multiprogramming system and running them on the separate processors of a multiprocessing system. This form of parallelism is commonly called multiple-instruction, multiple-data parallelism. MIMD computers generally fall into two categories:

- shared memory, and
- distributed memory.

This article discusses the limitations of the shared-memory and distributed-memory models for explicit parallel programming and examines a new model, Scientific Computing Associates' Linda parallel communication paradigm, which was designed specifically for parallel programming.[1] Then, this new model is adapted for use as the basis of a new class of operating system, and a specific instance, QIX, is presented. Like Linda, this new operating system model can support both the shared-memory and the distributed-memory styles of programming. Thus, it provides the benefits of both, while avoiding hardware dependencies. In addition, this new model provides benefits beyond those of either model and even those of Linda. For example, QIX incorporates a novel scheme for name resolution that is easier to use than other methods and provides significant benefits in the operating system. QIX also directly supports communication between programs written in different languages.

Current parallel programming models

To support multiprogramming, the following facilities were added to operating systems:

0018-9162/90/0200-0043$01.00 © 1990 IEEE

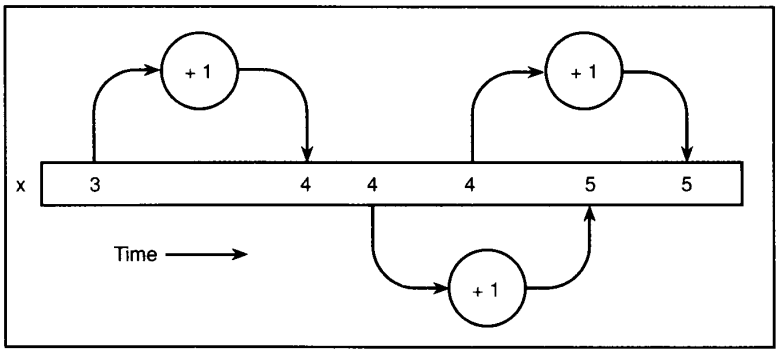


Figure 1. Updating an unprotected shared variable.

- interprocess communication,
- process synchronization,
- protection of shared resources, and
- interprocess name resolution.

Even though these facilities were originally developed to support multiprogramming, they have been adapted to try to fit the needs of explicit multiprocessing.

In existing operating systems, the above facilities are typically provided in one of two forms: the *shared-memory model* and the *distributed-memory model*.

In the shared-memory model, separate processes are allowed to share address space. Processes communicate by accessing and modifying shared data structures. Other mechanisms, such as semaphores, must be provided for synchronization and protection. Name resolution is typically performed statically, for example, at compile time by a link editor.

In the distributed-memory model, the processes do not share address space, and message passing is used for communication and synchronization. Access to shared resources is controlled through the use of server processes, which provide specific services to other processes. For example, name resolution is often provided by a name server.

Lauer and Needham[2] were the first to classify operating systems into these two models, calling them *procedure-oriented* and *message-oriented* systems. They also showed that these two models are duals of each other, in that any program (including an operating system) written using one model can be written as an equivalent program using the other model. They concluded that the choice of which model to use should be based mainly on the facilities (such as hardware facilities) on which the program is built. Given Lauer and Needham's conclusions, it is not surprising that shared-memory computers typically use shared-memory-model (procedure-call) operating systems and distributed-memory computers typically use distributed-memory-model (message-passing) operating systems.

In traditional multiprogramming, whether an operating system is based on the shared-memory or distributed-memory model has little effect on an applications programmer, since multiple processes are rarely used in a single application. Multiple processes are used only to run separate applications concurrently, and whatever small amounts of interaction there might be between applications can be handled using mechanisms such as pipes.

For explicit parallelism, however, the choice of model directly affects the applications programmer, because the operating system mechanisms that support communication, synchronization, protection, and naming are the ones used to write parallel programs. This means that when parallel computers are used for explicit parallelism, the difference between shared-memory and distributed-memory hardware is directly exposed to applications programmers.

To exploit parallelism in a single program, the programmer is forced to use a hardware-specific programming model and associated low-level mechanisms. This is analogous to programming using assembly language and has the same disadvantages: parallel programs are difficult to write because the programmer must deal with hardware details, and the resulting programs are not portable to other parallel (or even sequential) systems. The end result is that most parallel computers sold today are rarely used to write true parallel programs.

Shared-memory model. In this model, processes communicate via shared memory using normal memory read-and-write instructions. The familiarity of memory operations makes shared memory appear easy to use, but, unless other mechanisms are provided for synchronization and protection, the resulting programs can suffer from unwanted nondeterminism such as race conditions. Figure 1 shows an example of a race condition where the shared variable x is incremented three times but its value only increases by two because the third process read the old value of x before the second process wrote its updated value.

In systems built using a shared-memory model, mechanisms such as busy waits, semaphores, or monitors are typically used for protection and synchronization. For example, the variable x can be protected by use of a counting semaphore. Any process wanting to access x must first wait on the semaphore and then signal the semaphore when it is done. If another process tries to access x while the semaphore is locked, it will wait. The semaphore must be initialized before it can be used; the C++ code for this might look like:

```
// the shared variable
int x;
// semaphore for x
semaphore x_sem;
// initialize the semaphore
signal(x_sem);
```

The following code is then used to modify x:

```
// wait until x is not in use
wait(x_sem);
x = x + 1;
// free x
signal(x_sem);
```

Similar code can be used to decrement or otherwise modify x. Unfortunately, the need for synchronization is easily overlooked, especially when parallelizing existing programs. It is all too easy to put in a little protection and assume the program is correct if it runs properly a few times, just to have it fail sometime later. So, while programs may be relatively easy to write using the shared-memory model, getting them to run correctly or assuring that they will continue to run correctly can be difficult.

A further problem with mechanisms such as semaphores is the tendency to sequentialize a program. For example, consider a set of processes that spend most

of their time accessing a large shared data structure. If the entire structure is protected by a single semaphore, only one process can run at a time, causing execution of the processes to be sequentialized. Indeed, the overhead from using a semaphore will often cause this program to run slower on multiple processors than an equivalent program on a single processor. A common solution to this problem is to use multiple semaphores to protect subareas of the data structure; however, finding the correct granularity can be difficult.

In the shared-memory model, processes use shared address space for communication and a mechanism such as semaphores for synchronization and protection, but some other mechanism is needed to resolve names between separate processes. For example, in the semaphore program, separate processes must be able to access the same memory locations for the shared names *x* and *x_sem*.

Many parallel programming systems avoid adding a new naming mechanism by using the program linker to resolve shared names. (Sometimes this is handled by a separate utility — a configurer — but it is still performed at link time.) Static (link time) name resolution severely restricts parallel programs. For example, if communication paths are statically specified, then programs must be relinked when the number of processors or the communication topology changes. Static linking also means that all parts of a program that need to communicate must be linked at the same time. In some cases, such as communication between a user program and the operating system, this is problematic. Some parallel computers, especially those used as attached processors on a host, solve this problem by linking each application with the entire operating system (in this case the operating system is usually called a runtime system and the computer is typically rebooted for each application). Other systems use mechanisms such as jump tables or provide some form of dynamic (runtime) name service.

Distributed-memory model. The distributed-memory model uses message passing for communication and synchronization. Messages are sent to a specific message channel and received by waiting on that channel. There are many variations on this, for example, whether the sender blocks until the message is received (synchronous send) or not (asynchronous send), or how many senders and receivers are allowed to use the same channel. Another variation relates to how channels are specified. In many systems, a message is sent to a specific process on a specific processor (consequently, such a system can only have a single receiver per channel).

Writing programs for parallel machines is traditionally much harder than writing programs for sequential machines.

Shared resources (like the variable *x* in Figure 1) are protected by enclosing them inside a process that has sole access to the resource. Such a process is called a *server process*. Here, for example, is a process that receives messages to increment or decrement *x*.

```
x_process(channel chan_id)
{
  int x = 0;
  // loop forever receiving messages
  while(TRUE) {
    message msg;
    receive (chan_id, msg);
    switch (msg) {
    case INCREMENT:
      x = x + 1;
      break;
    case DECREMENT:
      x = x - 1;
      break;
    default:
      return;
    }
  }
}
```

Other processes must send a message to this process to increment the value of *x*:

```
message msg = INCREMENT;
send (chan_id, msg);  // increment x
```

The variable *x* is protected from simultaneous access because the *x_process* can only receive and process a single message at a time. But, the added complexity and overhead of having to encapsulate each shared resource in its own separate process pays for this safety. In addition, the process that protects a shared resource must anticipate every possible use of its resource. For example, in the above program there is no method of examining the value of the variable *x* or of modifying *x* by setting it to a specific value.

In the distributed-memory model, processes use message passing for communication and synchronization, and server processes for protection of shared resources. But, a mechanism is still needed to resolve names between separate processes. For example, in the message-passing program, a process wanting to increment the value of *x* must know the *chan_id* for the *x_process* in order to send a message to it. The most common way to do this is to provide a name server process.

A typical name service is provided by Berkeley Unix 4.3 to allow the socket mechanism to resolve names across a network. For example, to connect to the *whois* service on a remote processor, the following code could be used (error checking has been omitted for clarity):

```
// get the host entry for the
// remote machine named george
hostent *hp =
  gethostbyname("george");

// get service entry for whois
// service using tcp protocol
servant *serv =
  getservbyname("whois", "tcp");

// create a socket
int sock = socket (hp->h_addrtype,
  SOCK_STREAM, 0);

// connect to the remote service
sockaddr_in sa;
bcopy((char*)hp->h_addr, (char*)
  sa.sin_addr, hp->h_length);
sa.sin_family = hp->h_addrtype;
sa.sin_port = serv->s_port;
connect(sock, sa, sizeof(sa));

// at this point, sock is a socket that
// can communicate with
// the remote service.
```

The code to use this name service has been shown mainly to demonstrate how obscure it is. Not surprisingly, only the bravest programmers use Unix for writing parallel programs. This problem is not restricted to Unix; other name services are often just as bad, and add considerable complexity to parallel programs.

Choosing a model. Neither the shared-memory nor distributed-memory model is

clearly superior. In general, the shared-memory model is somewhat easier to program, but the resulting programs are more likely to contain unwanted nondeterminism, while the distributed-memory model is harder to program but more likely to run correctly. Some programs are easier to write using one model rather than the other, depending on factors such as the amount of communication and how much data can be treated as read-only. However, regardless of the model used, writing programs for parallel machines is traditionally much harder than writing programs for sequential machines.

When operating systems based on either the shared-memory or distributed-memory models are used for parallel programming, the choice of which model to use has been effectively taken away from the application programmer because the operating system supplies the primitives used for writing true parallel programs, and these generally mirror the underlying hardware. This means that a program written for one parallel machine is not easily portable to another, so the large investment spent writing a parallel program is generally lost when a different machine is purchased.

Linda

Linda is a parallel communication mechanism developed by David Gelernter at Yale University.[3] Rather than communicating with messages or through shared memory locations, processes communicate in Linda via a shared data space called tuple space. Tuple space acts something like an associative memory, since tuples are identified by matching on a key rather than using a specific address. There are four fundamental operations on tuple space:

out	place a tuple in tuple space
in	match a tuple and remove it from tuple space
rd	match a tuple and return a copy of it
eval	create an active tuple (a new process)

A tuple is an ordered collection of data items. For example, the tuple

```
("hello", 42, 3.14)
```

contains three data items: a string, an integer, and a float. The operation

```
out("hello", 42, 3.14);
```

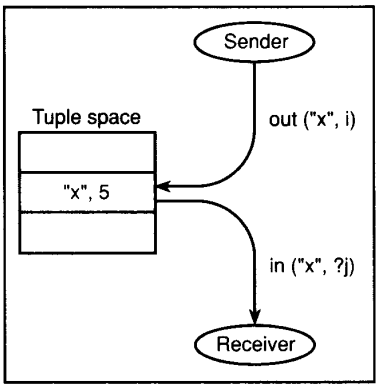


Figure 2. Message passing using Linda.

places this tuple into tuple space. The *out* operation never blocks, and there can be any number of copies of the same tuple in tuple space.

Tuples in tuple space are accessed by matching against a template. For example, the operation

```
int i; float f;
in("hello", ?i, ?f);
```

will match any tuple in tuple space with three fields whose first field is the string "hello," and whose second and third fields are an integer and a float, respectively. In this case, the string "hello" is the key used to find the matching tuple. If a matching tuple is found, the variables i and f are assigned the corresponding values from the matching tuple, and the tuple is removed from tuple space. If no matching tuple is found, the current process blocks. If the template matches more than one tuple, an arbitrary one is picked. The *rd* operation is like *in*, except that the matched tuple is not removed from tuple space.

The *eval* operation resembles *out*, except that it creates an active tuple. For example, if *fcn* is a function, then

```
eval("hello", fcn(z), true);
```

creates a new process to evaluate *fcn*, which proceeds concurrently with the process that made the *eval* call. When the new process completes, it leaves the resulting passive data tuple in tuple space, just like an *out* operation.

Linda is a high-level tool for parallel programming in that it is not directly based or dependent on a specific hardware mechanism. As with any higher level tool, there will always be some overhead associated with Linda, but it is not excessive (see the sections entitled "Implementing Linda" and "Kernel Linda"). This is similar to higher level languages compared with assembly languages, where any overhead is usually outweighed by the benefits of easier programming and portability. Likewise, the flexibility of Linda can more than make up for any overhead. For example, dynamic load balancing, where the processing load is dynamically adjusted between multiple processors, is rarely used in parallel programs because it is difficult to implement using message passing or semaphores, despite its potential performance improvement; but it is relatively easy to implement using Linda (an example of dynamic load balancing is given in a later section). Linda also makes it easy to experiment with the parallel structure of a program to increase its performance.

Using Linda like a shared-memory model. Linda can be used to model shared memory, but with the advantage that it is easy to avoid unwanted nondeterminism. In the example above, where the shared variable x was incremented by multiple processes, a semaphore was used to protect the shared variable from simultaneous access. Using Linda, we can represent a shared variable x as a single tuple whose key is the name x (we assume that no one else is using x as a key); to increment the variable x in tuple space, the following operations would be used:

```
in("x", ?i);
out("x", i+1);
```

Since the *in* operation removes the matching tuple from tuple space, any other process trying to access the value of x will block until the tuple is put back by the *out* operation.

Using tuple space as a shared data space is not limited to systems with shared memory; it will also work with distributed-memory computers. This makes programs written in Linda portable between shared-memory and distributed-memory systems.

Using Linda like a distributed-memory model. Linda can also support the message-passing style of programming. In Figure 2, a message is sent using an *out* operation and received using an *in* operation.

If the receiver gets ahead of the sender, it will block waiting on the next message. The sender is allowed to run faster than the receiver because the *out* operation is asyn-

chronous (it never blocks). As in any asynchronous message-sending situation, if the sender can get significantly ahead of the receiver, some sort of flow control must be used to avoid overflowing processor memory (an example of using flow control is given in a later section).

In Linda, the ordering of messages is not guaranteed. For example, if one process executes two *out* operations in the following order:

```
out("x", 1);
out("x", 2);
```

then a separate process doing an *in* operation on the key "*x*" may receive the tuples in either order. The order is often unimportant, but if not, an additional field in the tuple can be used to sequence the messages. For example, both the sender and receiver can keep a local sequencing variable, initially set to zero. The *out* and *in* operations in Figure 2 are changed to

```
// in the sender process
out("x", send_sequence++, i);

// in the receiver process
in("x", recv_sequence++, ?j);
```

and the correct order is guaranteed. If there are multiple senders or receivers, then the sequence variables can be shared by storing them in tuple space.

Linda provides decoupling between the sender and receiver of a message in both time and space. Messages are decoupled in time because the *out* operation is asynchronous and because messages travel via tuple space. Since messages can be held in tuple space, sending a message between two programs not running at the same time is even possible. This is crucial in an operating system, where operating system services must be able to communicate with user programs that typically were not written when the operating system was written.

Messages are decoupled in space because individual messages are identified by their contents. Neither the sender nor the receiver needs to know the other's identity or location. This allows communication in Linda to be more dynamic than in systems where the sender and receiver of a message must explicitly rendezvous. In particular, sending and receiving processes can be located on any processor, or even dynamically moved between processors, with no effect on the program. This important property of Linda facilitates writing parallel programs.

Unlike some message-passing systems, Linda is not limited to a single sender and receiver for a message stream. With the number of clients changing over time, this is useful for writing server processes, which receive messages from many different clients. In this case, if the client requires a message in return, it should identify itself in the message to the server so the server can respond to the correct client.

Kernel Linda is a version of Linda designed to be used for system-level programming, as the interprocess communication mechanism for a parallel operating system.

Implementing Linda. At first glance, it might appear that tuple matching requires searching tuple space for each operation, a process that would clearly be too slow for use on a parallel computer. Fortunately, a number of techniques are used to speed up the matching process. The goal of these techniques is to reduce the amount of searching necessary and make the remaining searching as efficient as possible.

The most common approach to making Linda more efficient is to use a preprocessor to reduce or eliminate runtime searching.[4] The preprocessor analyzes a Linda program to determine the patterns of tuple usage. For example, consider a Linda program that contains two processes. The first process contains a single *out* operation:

```
out("x", i);
```

and the second process contains a single *in* operation:

```
in("x", ?j);
```

where *i* and *j* are integer variables. Since the key "*x*" is a constant at compile time, the Linda preprocessor determines that this pair of Linda operations can be implemented as a queue between the variables *i* and *j*. On a shared-memory system, this could be realized as a shared queue protected by a semaphore; on a distributed-memory system, this could be realized as an asynchronous message send. Thus, no searching is done, and the most efficient implementation for the target architecture is used without changes to the program's source code. In this case, there is no overhead from the use of Linda.

Using a preprocessor is not always possible if Linda is used, for example, with interpreted languages such as Lisp, Postscript, Smalltalk, or Prolog, since tuple space usage can change while the program is running. Even with compiled languages, determining tuple space usage is not always possible at compile time if the programs that will be communicating cannot be preprocessed at the same time (in particular, for communication between the operating system and a user program).

A common technique used by Linda implementations that do not use a preprocessor is to restrict the key to a single field. In this case, hashing is used so the search can be performed (on average) in constant time. The performance of such an implementation is quite acceptable, since the overhead of a hashing function is reasonably small compared with the cost of data transfer on a typical system. Allowing but a single key in a tuple places some restrictions on Linda, but Linda's main advantages — including portability and decoupling in time and space — remain. For instance, all examples of Linda in this article need only a single key.

A third technique used to make Linda more efficient is to split tuple space into smaller pieces, for example, through the use of multiple tuple spaces; doing so, less space needs to be searched even if searching is required. This technique is also helpful for systems that use hashing, since smaller hash tables can be used.

Kernel Linda

Until now, Linda has been used primarily to support application-level programming. To use Linda for system-level programming, a number of modifications were made. This section discusses those modifications and shows how they benefit system-level programming.

Kernel Linda is a version of Linda designed to be used for system-level programming, as the interprocess communication (IPC) mechanism for a parallel operating system.[5] Kernel Linda was also designed for use as the runtime system for the original Linda. In this case, the Linda

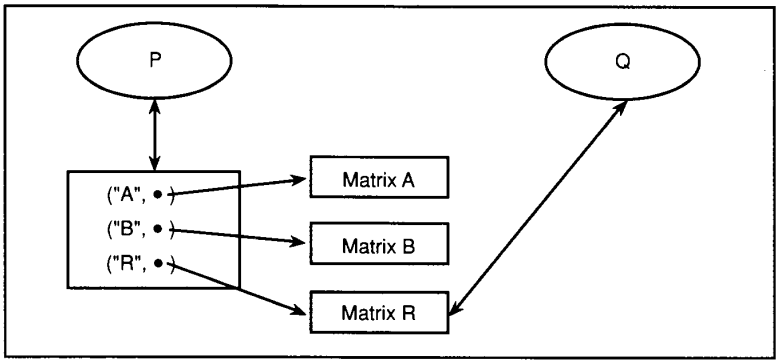


Figure 3. Tuple spaces as fields in a tuple.

preprocessor is used to generate calls to Kernel Linda, rather than to the normal Linda runtime library. This allows programs written in Linda to be integrated into an operating system based on Kernel Linda.

Since Kernel Linda is designed to support communication between operating system processes and user processes, a preprocessor cannot be used to speed up tuple space operations. Consequently, for efficiency, tuples in Kernel Linda are allowed to have only a single key. Except for the single-key restriction, Kernel Linda is a superset of Linda and includes the following extensions:

- multiple tuples spaces,
- the ability to store a tuple space as a field of a tuple, and
- a set of language-independent data types.

In addition, the ordering of tuples in tuple space is guaranteed in Kernel Linda. If one process executes the following *out* operations in the following order:

```
out("x", 1);
out("x", 2);
```

then a separate process doing an *in* on the key "*x*" is guaranteed to receive the values 1 and 2 in that order (assuming that no other Linda operations use the key "*x*" in this tuple space). This avoids the need for extra sequencing keys, and also means that Kernel Linda can be used to transmit a stream of ordered messages from one process to another.

Multiple tuple spaces. Kernel Linda incorporates a unique implementation of multiple tuple spaces that allows the tuple space name to be used somewhat like an additional key in a tuple. In traditional Linda, multiple keys in a tuple are often used to restrict communication to specific data structures; in Kernel Linda, this function can be more efficiently performed using multiple tuple spaces. The provision of multiple tuple spaces removes much of the need for multiple keys.

For example, in the *out* operation

```
// which row in the matrix
int RowNumber;
// data for the row
float data[SIZE];
out("MatrixA", RowNumber, data);
```

the constant *"MatrixA"* identifies a specific data structure, and the identifier *RowNumber* identifies a specific row in the matrix. A specific row of the matrix would be retrieved by searching on the first two keys:

```
int RowNumber = 42;
rd("MatrixA", RowNumber, ?data);
```

In Kernel Linda, the need for multiple keys can be eliminated by placing matrix A in its own tuple space, with the key being the row number. This also reduces the amount of searching required, since the name of the matrix is not used as a key during matching. In Kernel Linda, the above *out* operation would be written (in C) as

```
out(aMatrix, RowNumber, data);
```

where aMatrix is the name of a Kernel Linda tuple space. In C++, the special role of the tuple space is syntactically highlighted:

```
aMatrix.out(RowNumber, data);
```

The provision of multiple tuple spaces has other benefits for parallel programming. In a single global tuple space, name conflicts can occur between programs using the same key to identify tuples. Multiple tuple spaces allow communication to be more structured. In a multiapplication environment, some tuple spaces can be local to specific applications, while other tuple spaces can be common to some or all applications. This allows Kernel Linda to support both multiprogramming (separate applications running concurrently) and multiprocessing (single applications running on multiple processors).

Subsequent to our implementation of multiple tuple spaces in Kernel Linda, another design for multiple tuple spaces was done for Linda as part of the design of Linda 3, although it is not the type of system we describe here.[6]

Tuple spaces as tuple fields. Kernel Linda allows a tuple space to be used as a field in a tuple, which allows Kernel Linda to be used as its own name service. This is done by using a name as the key and the tuple space as the value of a tuple in another tuple space. No other naming mechanism is required. A tuple space can be used like a directory in a hierarchical file system, except that arbitrary graphs can be constructed. For example, all tuples spaces associated with an application can be given "names" by storing them in a tuple space, or one process can be given access to a tuple space owned by another application via a common tuple space. This is diagramed in Figure 3, where process P is computing with three matrices: *A*, *B*, and *R*, and a separate process Q has been given access to the *R* (result) tuple space.

Language-independent data types. Linda is traditionally implemented to work within a single language and borrows the data types from that language. Kernel Linda is designed to work with many different languages (both compiled and interpreted), so it provides a set of language-independent data types. For example, this allows a tuple space created by a C++ program to be accessed by a debugger or tuple space browser written in Postscript.

In traditional Linda, all fields in a tuple are passed by value. This is reasonable for small fields, such as numbers or even names. But, for large fields, such as structures or long character strings, this can result in extra copying. Passing some types by reference avoids this extra copying. More importantly, if a tuple space is to be

passed as a field in a tuple, it must be passed by reference, because two processes must have references to the same tuple space to communicate. Consequently, the Kernel Linda data types are divided into two groups, simple (pass-by-value) and composite (pass-by-reference). The simple types are

int	a 32-bit integer
real	a 32-bit floating-point number
real64	a 64-bit floating-point number
name	an atomic identifier
struct	a structure containing user-supplied data
null	the error value

The simple types are always passed by value. If a process passes one of these objects to another process (for example, via tuple space), each process will have a separate copy of the object, so modifying the object in one process will not affect the value of the object to the other process. The composite types are

dict	a Kernel Linda tuple space (dictionary)
string	a string of characters
block	an array of bytes
array	a heterogeneous array of other Kernel Linda objects

Composite types are always passed by reference; if a process passes one of these objects to another process (again, typically via tuple space), then both processes will have a reference to the same object, even if the processes are on different processors. Thus, modifying the object in one process will affect the value of the object to the other process. This is the only Kernel Linda communication mechanism: modifying a shared object. In particular, the Linda operations (*in*, *out*, and *rd*) modify shared dictionaries. Shared objects are implemented on a distributed-memory computer using a mechanism called locators, described in the next section.

Of course, it is possible to make a copy of a Kernel Linda object. Making a local copy of an object that was passed by reference gives the equivalent semantics as pass-by-value.

A dict (dictionary) is the Kernel Linda name for a tuple space. A tuple in a dictionary consists of a single key/value pair. The keys are almost always names or integers but can be of any data type. A value associated with a key in a dictionary can also be of any type, for example, another dictionary. Tuples are only allowed to have a single value, but this value can be a structured object, such as an array, which holds multiple values. As in Linda, multiple copies of the same tuple can be held in tuple space; this corresponds to a key in a Kernel Linda dictionary having multiple values.

Despite the fact that Kernel Linda does not use a preprocessor, the overhead compared to that of message passing is reasonable.

To allow the user to choose between value and reference semantics, strings and names are essentially the same type, except that strings are passed by reference and names are passed by value. The same is true of blocks and structs. This allows copying to be avoided, if desired. For example, names are typically used as keys in tuples, while strings are used for long strings of text such as shell scripts. Likewise, an executable load module is normally stored as a block to avoid making extra copies, but simple data structures, corresponding to a struct in C, are normally stored as a Kernel Linda struct.

Implementation. Kernel Linda evolved through several implementations, both on single-processor and distributed-memory computers. On a single-processor computer, and within a single processor on a distributed system, Kernel Linda is implemented as a shared-memory-model program. Internally, composite objects are protected by a semaphore to guard against concurrent updates. This semaphore is controlled by the system and is not visible to the user.

The implementation on a distributed-memory computer, where a distributed-memory-model program is used, is much more interesting. In this case, composite objects are manipulated using a form of distributed pointer called a *locator*, which, in addition to other information, contains the processor ID of the processor on which the object is located. When an operation is to be performed on a remote object, a message is sent to the processor containing that object, and a server process on the remote processor performs the operation. When an operation is to be performed on a local object (including operations by the server process after receiving a request from a remote processor), the shared-memory-model program is used. The implementation of Kernel Linda on a shared-memory computer is more straightforward than the one for the distributed-memory computer because it avoids the remote server process.

Despite the fact that Kernel Linda does not use a preprocessor, the overhead compared to that of message passing is reasonable. This overhead arises mainly from evaluation of the hashing function, any required searching within the hash table, and the locking of a semaphore. For transfers of reasonable amounts of data the overhead is about 10 percent, rising to as much as 50 percent for shorter transfers (a synchronization with no data transfer being the worst case). This is comparable to the overhead of higher level languages such as C, and, of course, when this overhead is unacceptable, the user can drop down into machine primitives, analogous to the way C programmers can drop down into assembly language.

QIX

QIX (pronounced "quicks") is a parallel, dynamically reconfigurable, server-based operating system implemented on a distributed, message-passing computer based on the Inmos transputer. Further implementations on other machines, including shared-memory computers or even single-processor computers, would also be reasonable.

To the user, QIX appears almost identical to Unix but, below the surface, it is considerably different. In terms of existing versions of Unix, QIX is most like Mach,[7] except that Mach is based on a message-passing model, while QIX uses Kernel Linda. Like Mach, QIX has a small operating system kernel that is replicated on each processor. The QIX kernel provides the lowest level of operating system facilities:

- memory allocation,
- process creation, scheduling, and synchronization,
- device drivers, and
- Kernel Linda.

The remainder of the operating system is implemented by servers, which communi-

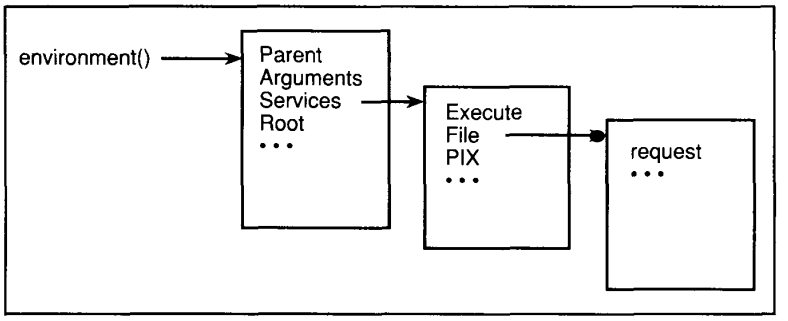


Figure 4. Accessing system services via Kernel Linda.

cate with each other using Kernel Linda. These servers implement a high degree of compatibility with Unix System V, with one significant exception: the process model.

As mentioned earlier, the Unix process model, which was developed for pseudo parallelism, is largely unsuitable for true parallel programming. Parallel computers that run Unix typically supply a second process creation mechanism for lightweight processes, resulting in two independent, incompatible mechanisms, one for parallel programs and one for compatibility with Unix. Put another way, these operating systems provide one process creation mechanism for pseudo parallelism (multiprogramming) and a separate mechanism for true parallelism (multiprocessing).

For true parallel programming, it is very important that process creation be as inexpensive as possible. Unix process creation, based on the fork and exec primitives, is relatively costly because a copy of the image space of a process is made, file descriptors are duplicated, and so on. The fork primitive is especially impractical on a distributed-memory computer, since standard optimizations such as copy-on-write cannot be used.

Those of us who developed QIX at Cogent Research wanted a single process creation mechanism that would serve both true and pseudo parallelism, while maintaining as much compatibility with Unix as possible within the absolute requirement for efficiency. We also wanted it to be easy to use and thus encourage programmers to write parallel programs. Once again, Linda proved invaluable, and the resulting process model is both simple and general. Indeed, the QIX process model would be suitable for use in a version of Unix designed for single-processor machines.

Process environments. In Unix, each process executes within an environment, but the information that constitutes an environment is not embodied in a single object. Access to information in a Unix process environment is through several different mechanisms or, in some cases, is not possible. Using Kernel Linda, we were able to encapsulate the process environment into a single object, which can be manipulated using normal Kernel Linda operations.

Each process in QIX has a dictionary (tuple space) associated with it called its environment. The environment dictionary contains all information about a process and is the only access the process has to the outside world. Heavyweight (Unix-like) processes will naturally have quite a bit of information in their environment, while lightweight processes can have very simple environments. The time it takes to create a new process depends on how much information is put into its environment dictionary; anything from extremely lightweight (an empty environment) to extremely heavyweight (even heavier than Unix!) processes can be created, or anything in between.

The environment of a Unix-style heavyweight process typically contains the following keys:

Parent: the environment dictionary of the parent process
Exit: the exit code of this process
Signal: where signals are sent
Arguments: the command line arguments (argc and argv)
Variables: Unix environment variables
File Descriptors: an array of open file descriptors for this process
Creation Mask: default file creation permissions
Process ID: the process ID of this process
Process Group: the current process group ID
Processor ID: the processor this process is running on
Working Directory: the current working directory
Real IDs: the real user ID of this process
Effective IDs: the current effective user ID of this process
Root: for communication between unrelated programs
Services: a dictionary containing the O/S servers

Since the process environment is a Kernel Linda dictionary, it can be manipulated using normal Kernel Linda operations. Typical Unix operations on processes can be performed using normal Linda operations on the process environment. For example, to wait until a process terminates and return its exit code, an *rd* operation can be done on the key *"Exit"* in the environment dictionary for the process:

```
env.rd("Exit", exit_code);
```

Services. The *"Services"* key in the environment is particularly significant. The value of this key is another dictionary that contains keys for all of the operating system services. On a typical system, the *"Services"* dictionary contains the following keys:

File: the file system
Pipe: Unix pipe service
TTY: serial ports
Execute: the execution service
Console: the system error console
PIX: the parallel interactive executive window system
Graphics: the graphics server
Window: terminal emulator windows
FTP: file transfer service
Tape: file backup
Null: the null device (like/dev/null)

Processes gain access to operating system services by using Kernel Linda as a name service. For example, as Figure 4 shows, to find the file service, a process looks in its environment for the key *"Services"* (using *rd*), and then in this dictionary for the key *"File."* The value of this key is another dictionary: the dictionary for the file service.

Once the file server dictionary is located, it is not necessary to walk this dic-

tionary chain again. A process communicates with the file system simply by doing Kernel Linda *in* and *out* operations on this dictionary. The file system server process normally sits blocked doing an *in* operation on the key *"request"* in its dictionary so, when a process does an *out* to this key, the server wakes up and performs the service. For the file service, the *out* operations contain requests for read and write operations on files and the *in* operations return the results of the file operations. Other operating system services perform similarly. A programmer can perform Linda operations to communicate with system services, if desired, but more typically access to these services is through a library, which translates standard Unix calls into the appropriate Kernel Linda operations. Thus, to the user, the file system is identical to Unix while, to the operating system, the files are handled by a server represented by a dictionary.

Normally, all (heavyweight) processes share the same Services dictionary but, because the Services dictionary is just another dictionary, adding new services or replacing individual services on either a systemwide or a process-by-process basis is possible, using normal Kernel Linda operations. It is also possible to move services between processors, or to replicate servers on multiple processors for added performance; for example, there is a separate instance of the pipe service on each processor. Many servers, including the file service, are stateless; it is possible to change or even completely replace these services while the operating system is running.

The same Linda debugging tools used to debug parallel programs can be used to browse around inside the operating system. For example, when writing this article, I used the Kernel Linda browser to see what services were available on my system.

Process creation. The use of Kernel Linda makes process creation in QIX easy (and quick!).[8] The same operations that create the environment for the new process also set up the communication paths for it.

- The parent process creates a Kernel Linda dictionary (using the *createdict* call), and then adds as much information to this dictionary as desired. Heavyweight processes will have lots of information placed in their environment dictionary, while lighter weight processes will have less information. Users can either place information in this dictionary explicitly (using the normal Kernel Linda operations) or use library calls to create standard environments. Communication paths between different processes in an application are created at the same time.
- A new process is then created, using the *execute* call. The program can be read from a disk file, but if multiple copies of a process are to be created (as for a processor farm), then it is more efficient to load an executable program from disk into a Kernel Linda block. The execute call can then be used to create multiple copies of the process quickly, since no further disk activity is required.
- The *execute* call can optionally take a processor ID that indicates the processor to be used to execute the new process. If the user does not supply a processor ID, then the execution service (accessed using the *"Execute"* key in the *"Services"* dictionary) supplies an appropriate processor by consulting the Oracle. The Oracle is yet another Kernel Linda dictionary found in the dictionary for the execution service. The Oracle contains information on loading and memory usage for all processors in a system and also contains a method (which can be overridden) for choosing a processor on which to run a new process. The default method chooses a processor with enough memory to run the new process and which also has the lowest load.

The use of Kernel Linda makes process creation in QIX easy (and quick!). The same operations that create the environment for the new process also set up the communication paths for it.

The QIX process creation mechanism also solves a problem with the *eval* operator in traditional Linda. The *eval* operator is used to create new processes. For example, if *fcn* is a function, then

```
eval("hello", fcn(z), true);
```

creates a new process to evaluate *fcn*. Unfortunately, the semantics of the *eval* call are not well defined, because the environment of the new process is not specified. For example, if *fcn* uses any global variables, they might or might not be shared with the parent process or, if the tuple passed to *eval* contains more than one function to be evaluated, they might or might not share address space. Thus, in a conventional Linda system, the semantics of an *eval* operation will usually be very different on a shared-memory system as opposed to a distributed-memory system, making *eval* nonportable. In QIX, the environment for the new process is explicitly specified.

Lightweight process creation. The following is an example of a program that creates a new process from a program on disk and communicates with it. The environment for the new process contains only two keys, so creating it is very fast. Here is the code to spawn the new process:

```
// the new environment dictionary
Val new_env = createdict();
// the value of x is the integer 5
new_env.out("x", (Val) 5);
// the value of y is the integer 20
new_env.out("y", (Val) 20);

// read the executable file from disk
Val proc = createblockfromfile("foo");

// create a new process
int pid = new_env.execute(proc);

// collect results
Val result;
// read the value of z
new_env.in("z", result);
printf("z = %d\n", (int) result);
```

A *Val* is a Kernel Linda value; in C++, all Kernel Linda objects are of C++ type *Val* but, as discussed before, these objects also have a Kernel Linda type. In the above program, the *Val new_env* is of Kernel Linda type dict; *x*, *y*, and *z* are of Kernel Linda type int; and *proc* is of Kernel Linda type block. Here is the program for the new process to be created:

```
// the process to be created
void main()
{
  // access my environment
```

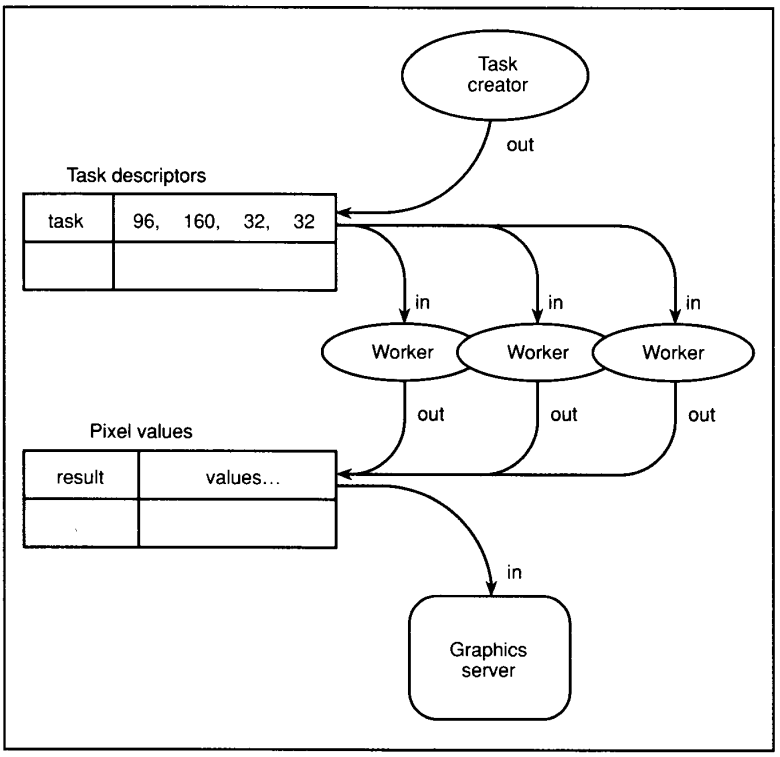


Figure 5. Processor farm using Kernel Linda.

```
    Val env = environment();
    Val x, y;
    // read the value of x and y
    env.rd("x", x);
    env.rd("y", y);
    int z = (int) x + (int) y;
    // set the value of z
    env.out("z", (Val) z);
}
```

This program is compiled like any C++ program and placed in a file named "foo." In this example, the values of the keys are integers to be added together; typically, a much more complex calculation would be performed by the spawned process.

In creating this process, we used the environment dictionary for communication between the parent process and its child. Note that because the new environment has no *"Services"* dictionary, the child process cannot access any operating system services beyond those in the Kernel. In particular, it must pass any results back to its parent, rather than printing them itself. To provide other operating system services, we need to create a more heavyweight process.

Heavyweight process creation. One way to create a more heavyweight process is to explicitly place all desired keys in the environment dictionary; there are a number of shortcuts for doing this. For example, the *ecopy* call makes a copy of the current process environment suitable for use as the environment for a child process, so a heavyweight process can be created as follows:

```
Val prog =
  createblockfromfile("foo");
Val new_env =
  environment().ecopy();
int pid = new_env.execute(prog);
```

Finally, there is a forkexecvp call that emulates the most common usage of the Unix fork and exec calls:

```
// command line arguments
char **argv;
// file descriptors
int *fds;
// optional processor id
int processor;
int pid = forkexecvp("foo",
  argv, fds, processor);
```

Parallel application programming in QIX

In QIX, as in other operating systems, the same mechanisms for parallelism used by the operating system are used for writing parallel application programs; but, in this case, the applications programmer can use Kernel Linda. This section gives a fairly complete example of how to use QIX to write a true parallel program. This example uses a form of parallel program called a processor farm, although many other forms are also possible. In a processor farm, multiple worker processes perform tasks assigned by a master (or task creator) process. The number of workers will usually vary according to the number of processors in the system.

This example is applicable to graphics applications such as ray tracing, image processing, or computation of the Mandelbrot set, where there is an image to be computed and subareas of this image are portioned out as tasks to the various workers. Figure 5 diagrams the example.

The task descriptors define the area of the image to be computed; for example,

```
( 96, 160, 32, 32 )
```

defines a rectangular area of the image at x position 96, y position 160, and a width and height of 32 picture elements. This task descriptor is implemented as a C++ struct:

```
struct task_descriptor {
  int x, y, xsize, ysize;
} td;
td.xsize = td.ysize = 32;
```

The task creator process places the task descriptors into its environment with a loop containing an *out* primitive:

```
Val env = environment();
for (td.y = 0; td.y<y_image;
  td.y += td.ysize)
    for (td.x = 0; td.x < x_image;
      td.x += td.xsize){
      Val tdval = createstruct(
        sizeof(td), (char*)&td);
    env.out("task", tdval);
  }
```

Each worker process first gets a task descriptor from the environment of its parent. The worker then computes that part of the image, sends out the resulting pixels, and repeats the procedure until there is no more work to be done.

```
Val parent;
environment().rd("Parent", parent);
Val task; // to receive task descriptors
Val result;  // for resulting pixels
while(1) {
   parent.in("task", task);
   task_descriptor* tdp = task.value();
   // compute area of image
   result = compute(tdp->x, tdp->y,
      tdp->xsize, tdp->ysize);
   parent.out("result", result);
}
```

Sending task descriptor messages via tuple space means the task creator process need not be concerned with details about where the workers are located, nor even with how many workers there are. The number of workers can change even while the program is running.

Flow control. In the processor farm example, no flow control exists between the task creator process and the workers, so it is possible for the task creator to overflow memory. If we are creating only a few hundred task descriptors, then this would not usually be a problem; but, if it is a problem, flow control is relatively easy to implement in Linda.

For example, the task creator can initially send out a fixed number of task descriptors (typically twice the number of worker processes) and then block doing an *in* operation on the key *"ack"* in its environment. As each worker *in*'s a task descriptor, it also does an *out* operation to the key *"ack."* When the task creator receives an ack, it sends out a new task descriptor. Thus, the number of task descriptors remains fixed.

Dynamic load balancing. To achieve optimal processor use, the processor farm model of parallel computation requires that each worker perform about the same amount of work. Unfortunately, work rarely distributes that evenly; one or two tasks will usually take much longer than others, causing the other workers to wait while those tasks complete. This is especially true in graphics applications such as the ones considered above, where some tasks can take orders of magnitude longer than others.

Using Linda, it is relatively easy to solve this problem. If one worker discovers that it has a particularly difficult area of the image to compute, it can subdivide its task into smaller tasks and send the new tasks off to other workers. As Figure 6 shows, workers do this by performing an *out* operation to the key *"task"* in the environment of the task creator process (the same as the task creator would). Since messages are sent via tuple space, workers do not care whether the task descriptor they receive was produced by the task creator or another worker.

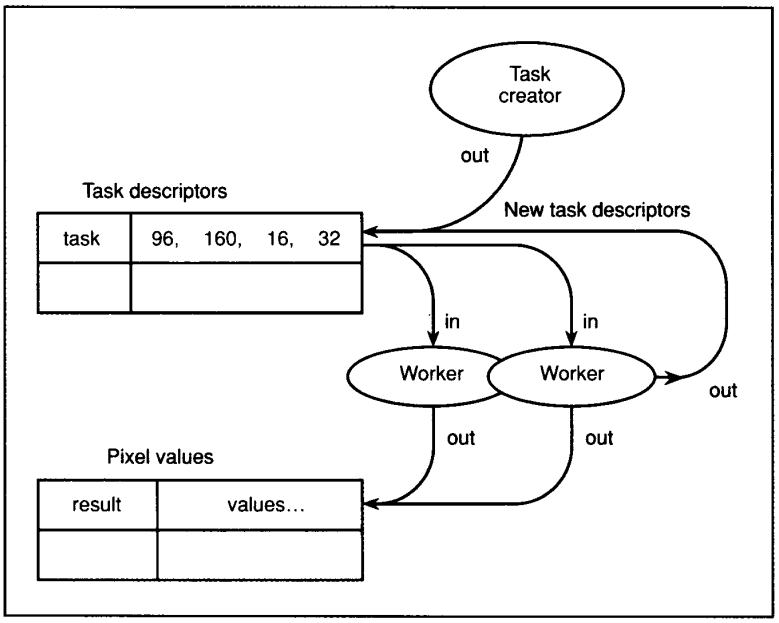


Figure 6. Dynamic load balancing in Kernel Linda.

How workers decide whether their task is too complicated depends on the application. For example, in ray tracing, you might decide that if an area of the image contains more than a set number of objects, the area will be subdivided. Or for a Mandelbrot program, the black areas of the image are the most costly to compute, so we could keep track of the percentage of black points that have been computed and, if the percentage exceeds a certain threshold, subdivide the area. The point at which a task is subdivided can even change dynamically.

An alternative way to structure a program that uses dynamic load balancing is to eliminate the task creator process entirely, and initially create a single worker with a single task descriptor for the entire job. If the task is simple enough, that process can easily complete it without going to the trouble of creating any other workers. If the task is not that simple, then a new worker and task descriptor can be created, and so on recursively. Thus, the amount of parallelism can adapt to the task at hand. This eliminates creating processes that will have little or nothing to do, for example, creating hundreds of worker processes to create a blank image. When the number of workers and available processors is roughly the same, load balancing can still be done by subdividing task descriptors, but without creating new worker processes.

PIX. One of the best ways to determine the power and efficiency of a new software paradigm is to actually build a reasonably large application with it. To test the ideas in Kernel Linda, we decided to write a window system, which is a substantial project. We also decided that, like the NeWS window system from Sun Microsystems and NextStep user interface from Next, we would base this window system on the Postscript graphics language that Adobe Systems, Inc. developed. Writing a Postscript interpreter itself is no small task; writing a Postscript interpreter that is fast enough for interactive use is harder still; and the conventional wisdom says that writing a parallel program is harder than writing a sequential program, so we decided that writing a parallel interactive Postscript interpreter would be a good trial.

The resulting window system, PIX (parallel interactive executive), is in daily use on a distributed computer (the Cogent Research XTM), and is certainly fast enough for interactive use. Linda handles all communication, even down to tracking

mouse movement. Total development time was about 2 person-years, although we felt the choice of C++ as the implementation language also had a favorable effect on development time.

Also interesting was how the use of Kernel Linda allowed us to significantly improve the window system design. For example, one of the more innovative features of the NeWS window system is the addition of quasi-parallelism to Postscript. New Postscript processes can be created, and they communicate using two different mechanisms: shared memory protected by monitors, and a message-passing system called events. NeWS uses this parallelism to deal with the inherent concurrency of human-computer interaction. But, in NeWS, this parallelism is simulated inside a single Unix process because Unix process creation is so expensive, and Postscript processes run nonpreemptively (each process runs to completion unless it explicitly gives up control). This can be disconcerting to the window system user, since windows that are supposed to be executing in parallel often don't, due to time-consuming processes effectively locking out any other user interaction.

Under QIX, we were able to implement PIX as a true parallel program. In PIX, the Postscript processes are individually scheduled processes, and they communicate using Kernel Linda. To do this, we added a Kernel Linda binding to Postscript, using Postscript dictionaries as tuple spaces. Kernel Linda eliminates the need to use dangerous shared memory and replaces the complicated and slow NeWS event-matching process.[9] We found that using Linda not only makes programming user interfaces in Postscript much easier, but also results in a true parallel user interface that can run on multiple processors. In particular, PIX can reconfigure itself to take advantage of new processors as they are added to a system, so the user interface can distribute along with an application. Thus, the user interface does not become a bottleneck.

Kernel Linda is a variant of Linda, designed specifically to support system-level programming on parallel computers. The use of Kernel Linda was a significant factor in the design and implementation of the QIX operating system. QIX is neither a shared-memory or distributed-memory model operating system, but combines many of the best features of both, with some significant added features, such as a simple and powerful naming mechanism.

The QIX operating system provides a powerful environment that makes it easier to write parallel programs, featuring

- Unix-compatible system calls and utilities,
- familiar languages,
- Linda extensions, and
- a powerful user interface.

Using Linda makes it easy to write programs that are independent of architecture or topology. Indeed, because the QIX operating system, PIX window system, and other utilities are written using Kernel Linda, they automatically reconfigure themselves to take advantage of new processors as they are added to a system.

Linda is a high-level tool. It allows the programmer to concentrate on writing parallel programs and leaves the details to the operating system.

In addition to supporting the writing of true parallel programs, QIX also supports traditional multiprogramming. Since multiprogramming increases hardware utilization by allowing several users or programs to share processor resources, and multiprocessing increases user productivity by decreasing program turnaround time, QIX helps maximize utilization of both computing and programmer resources. ■

Acknowledgments

Realization of the ideas in this article involved an enormous effort by a handful of people at Cogent Research. In particular, the design of Kernel Linda evolved through successive implementations by Hermi Singh. The QIX operating system was developed primarily through a Herculean effort by Tom Merrow, and many of the ideas on incorporating Kernel Linda into it are his. Implementation of a window system that runs in parallel and its associated Postscript interpreter was performed by Philip Goward. Marta Kallstrom wrote a wonderful Linda browser and other utilities. John Roberts and Steve Splonskowski provided the other tools necessary for a complete operating system. Through it all, Charles Vollum provided the vision and foresight that made all this work possible.

References

1. D. Gelernter, "Getting the Job Done," *Byte*, Vol. 13, No. 11, Nov. 1988, pp. 301-308.
2. H.C. Lauer and R.M. Needham, "On the Duality of Operating Systems Structures," *Proc. Second Int'l Symp. Operating Systems*, IRIA, Oct. 1978. Reprinted in *Operating Systems Review*, Vol. 13, No. 2, Apr. 1979, pp. 3-19.
3. S. Ahuja, N. Carriero, and D. Gelernter, "Linda and Friends," *Computer*, Vol. 19, No. 8, Aug. 1986, pp. 26-34.
4. N. Carriero, *Implementing Tuple Space Machines*, doctoral dissertation, Yale University, 1987.
5. *Kernel Linda Specification*, Cogent Research, Inc., technical note 89.16.
6. D. Gelernter, "Multiple Tuple Spaces in Linda," *Proc. Parallel Architectures and Languages Europe* (Parle 89), Vol. 2, Springer-Verlag LNCS 366, June 1989, pp. 20-27.
7. M. Accetta et al. "Mach: A New Kernel Foundation for Unix Development," *Proc. Atlanta Usenix Conf.*, June 1986, pp. 93-112.
8. *Process Creation in QIX*, Cogent Research, Inc., technical note 89.3.
9. W. Leler, "PIX: The Latest NeWS," *Proc. 34th IEEE Computer Society Int'l Conf.*, (Compcon Spring 89), 1989, CS Press, Los Alamitos, Calif., Order No. 1909, pp. 239-242.



Wm Leler was employed from 1987 to early 1990 at Cogent Research, Inc., in Beaverton, Oregon, and is currently pursuing independent projects. He is also an adjunct faculty member in the Computer Science and Engineering Department of the Oregon Graduate Institute. His research interests include computer graphics, parallel computation, and languages. He wrote the book *Constraint Programming Languages: Their Specification and Generation* and is a member of the IEEE Computer Society and the Association for Computing Machinery.

Leler received undergraduate degrees in fine arts and electrical engineering from Rice University in 1978 and a PhD in computer science from the University of North Carolina in 1987.

Readers may contact Leler by mail at 2950 NW Imperial Terrace, Portland, OR 97210, or by e-mail at wm@cse.ogi.edu.